Module-XI

Lecture-I and II

Built in Self Test

1. Introduction

Till now we have been looking into VLSI testing, only from the context where the circuit needs to be put to a “test mode” for validating that it is free of faults. Following that, the circuits tested OK are shipped to the customers with the assumption that they would not fail within their expected life time; this is called off-line testing. In other words, in off-line testing, a circuit is tested once and for all, with the hope that once the circuit is verified to be fault free it would not fail during its expected life-time. However, this assumption does not hold for modern day ICs, based on deep sub-micron technology, because they may develop failures even during operation within expected life time. To cater to this problem sometimes redundant circuitry are kept on-chip which replace the faulty parts. To enable replacement of faulty circuitry, the ICs are tested before each time they startup. If a fault is found, a part of the circuit (having the fault) is replaced with a corresponding redundant circuit part (by re-adjusting connections). Testing a circuit every time before they startup, is called Built-In-Self-Test (BIST). In this module we will study details of BIST. Once BIST finds a fault, the readjustment in connections to replace the faulty part with a fault free one is a design problem and would be not be discussed here.

2. Basic architecture of BIST

As discussed in the last section, BIST is basically same as off-line testing using ATE where the test pattern generator and the test response analyzer are on-chip circuitry (instead of equipments). As equipments are replaced by circuitry, so it is obvious that compressed implementations of test pattern generator and response analyzer are to be designed. The basic architecture of BIST is shown in Figure 1.

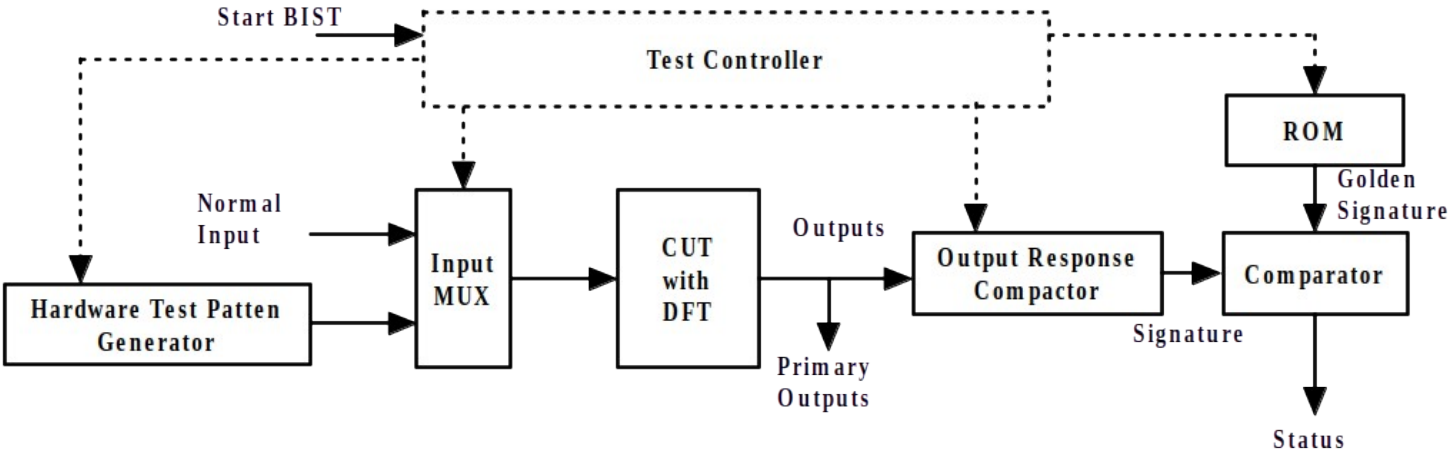

Figure 1. Basic architecture of BIST

As shown in Figure 1, BIST circuitry comprises the following modules (and the following functionalities)

1. *Hardware Test Pattern Generator:* This module generates the test patterns required to sensitize the faults and propagate the effect to the outputs (of the CUT). As the test pattern generator is a circuit (not equipment) its area is limited. So storing and then generating test patterns obtained by ATPG algorithms on the CUT (discussed in Module XI) using the hardware test pattern generator is not feasible. In other words, the test pattern generator cannot be a memory where all test patters obtained by running ATPG algorithms (or random pattern generation algorithms) on the CUT are stored and applied during execution of the BIST. Instead, the test pattern generator is basically a type of register which generates random patterns which act as test patterns. The main emphasis of the register design is to have low area yet generate as many different patterns (from 0 to 2^n, if there are n flip-flops in the register) as possible.

2. *Input Mux:* This multiplexer is to allow normal inputs to the circuit when it is operational and test inputs from the pattern generator when BIST is executed. The control input of the multiplexer is fed by a central test controller.
3. *Output response compactor:* Output response compacter performs lossy compression of the outputs of the CUT. As in the case of off-line testing, in BIST the output of the CUT is to be compared with the expected response (called golden signature); if CUT output does not match the expected response, fault is detected. Similar to the situation for test pattern generator, expected output responses cannot be stored explicitly in a memory and compared with the responses of the CUT. So CUT response needs to be compacted such that comparisons with expected responses (golden signatures) become simpler in terms of area of the memory that stores the golden signatures.
4. *ROM:* Stores golden signature that needs to be compared with the compacted CUT response.
5. *Comparator:* Hardware to compare compacted CUT response and golden signature (from ROM).
6. *Test Controller:* Circuit to control the BIST. Whenever an IC is powered up (signal start BIST is made active) the test controller starts the BIST procedure. Once the test is over, the status line is made high if fault is found. Following that, the controller connects normal inputs to the CUT via the multiplexer, thus making it ready for operation.

Among the modules discussed above, the most important ones are the hardware test pattern generator and the response compactor. The other ones are standard digital blocks [1]. In the next two sections we will discuss these two blocks in details.

3. Hardware pattern generator

As discussed in the last sub-section, there are two main targets for the hardware pattern generator—(i) low area and (ii) pseudo-exhaustive pattern generation (i.e., generate as many different patterns from 0 to 2^n as possible, if there are n flip-flops in the register). Linear feedback shift register (LFSR) pattern generator is most commonly used for test pattern generation in BIST because it satisfies the above two conditions. There are basically two types of LFSRs, (i) standard LFSR and (ii) modular LFSR. In this section we will discuss both these LFSRs in detail.

3.1 Standard LFSRs

Figure 2 shows an external exclusive-OR also called standard LFSR. The circuit representation of standard LFSR is shown in Figure 3. LFSR, as the name suggests, it is basically a shift register having D flip-flops where the output of the last flip-flop provides feedback to the input of the first flip-flop. If there are *n* flip-flops (numbered as $X_0, X_1 ... X_{n-1}$), then the LFSR is called *n*-stage LFSR. The feedback is basically a linear XOR function of the outputs of the flip-flops. Output of any flip-flop may or may not participate in the XOR function; if output of any flip-flop X_i say, provides input to the XOR function then corresponding tap point h_i (Figure 2) is 1. Similarly, if output of flip-flop X_i does not provide input to the XOR function then corresponding tap point h_i (Figure 2) is 0. In the circuit representation (Figure 3) if h_i=0, then there is no XOR gate in the feedback network corresponding to the output of the flip-flop X_i; otherwise, the XOR gate is included.

A properly-designed LFSR can generate as a near-exhaustive set of patters, as it can cycle through distinct $2^n - 1$ states (except 0s is all flip-flops). Such a properly designed LFSR is known as a maximal length LFSR.

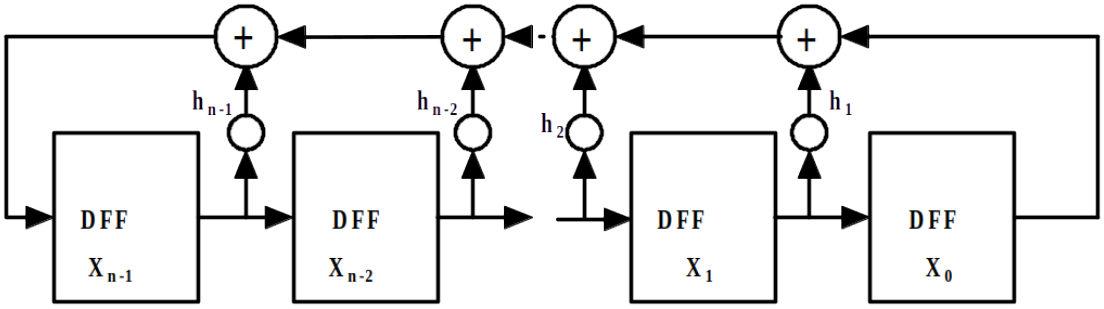

Figure 2. Standard LFSR

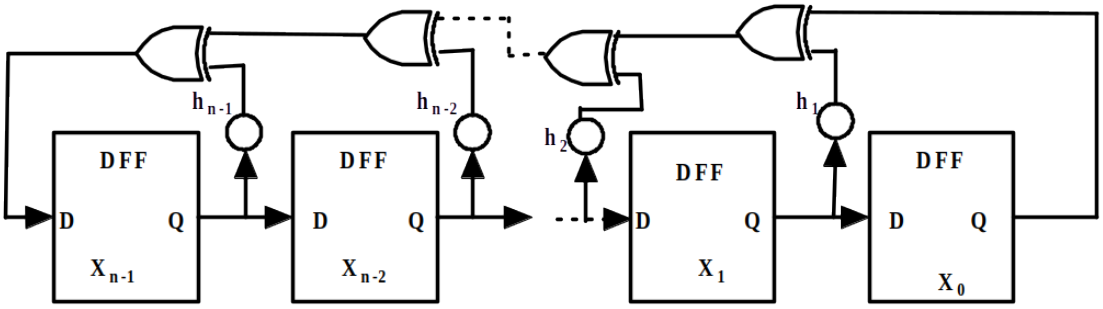

Figure 3. Circuit representation of standard LFSR

The following matrix system of equations describes the standard LFSR.

$$\begin{bmatrix} X_0(t+1) \\ X_1(t+1) \\ \cdots\cdots\cdots\cdots \\ X_{n-3}(t+1) \\ X_{n-2}(t+1) \\ X_{n-1}(t+1) \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0.....0 & 0 \\ 0 & 0 & 1.....0 & 0 \\ \cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots & \\ 0 & 0 & 0.....1 & 0 \\ 0 & 0 & 0.....0 & 1 \\ 1 & h_1 & h_2 \quad h_{n-2} & h_{n-1} \end{bmatrix} \begin{bmatrix} X_0(t) \\ X_1(t) \\ \cdots\cdots\cdots\cdots \\ X_{n-3}(t) \\ X_{n-2}(t) \\ X_{n-1}(t) \end{bmatrix}$$

This LFSR in terms of the matrix can be written as $X(t+1) = T_S X(t)$. It may be noted the matrix T_S defines the configuration of the LFSR. Leaving behind the first column and the last row T_S is an identity matrix; this indicates that X_0 gets input from X_1, X_1 gets input from X_2 and so on. Finally, the first element in the

last row is 1 to indicate that X_{n-1} gets input from X_0. Other elements of the last row are the tap points $h_1, h_2, \ldots h_{n-2}, h_{n-1}$. The value of $h_i = 1, \ (1 \le i \le n-1)$, indicates that output of flip-flop X_i provides feedback to the linear XOR function. Similarly, the value of $h_i = 0, \ (1 \le i \le n-1)$, indicates that output of flip-flop X_i does not provide feedback to the linear XOR function.

This LFSR can also be described by the characteristic polynomial:

$$f(x) = 1 + h_1 x + h_2 x^2 + \ldots h_{n-2} x^{n-2} + h_{n-1} x^{n-1} + x^n$$

Now, we give an example of a standard LFSR and illustrate the pattern generated the register. Figure 4 shows an example of a standard LFSR.

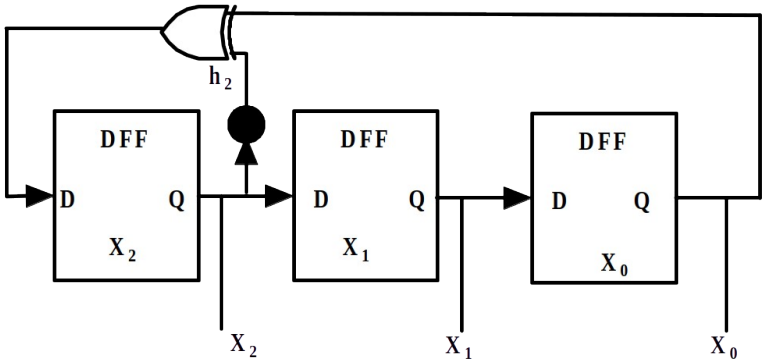

Figure 4. Example of a standard LFSR

The matrix of the LFSR is as follows

$$\begin{bmatrix} X_0(t+1) \\ X_1(t+1) \\ X_2(t+1) \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_0(t) \\ X_1(t) \\ X_2(t) \end{bmatrix}$$

It may be noted that output of flip-flop X_2 provides feedback to the XOR network, while flip-flop X_1 does not; so $h_1 = 0$ and $h_2 = 1$. The characteristic polynomial of the LFSR is $f(x) = 1 + x^2 + x^3$.

If the initial values of the flip-flops are $X_0 = 1, X_1 = 0, X_2 = 0$ then the sequence of patters is as follows:

$$\begin{bmatrix} X_0 \\ X_1 \\ X_2 \end{bmatrix} ::: \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

So the LFSR generates 7 patterns (excluding all 0s) after which a pattern is repeated. It may be noted that this LFSR generates all patters (except all 0s) which are generated by a 3 bit counter, however, the area of the LFSR is much lower compared to a counter. In a real life scenario, the number of inputs of a CUT is of the order of hundreds. So LFSR has minimal area compared to counters (of order of hundreds).

3.2 Modular LFSRs

Figure 5 shows an internal exclusive-OR also called modular LFSR. The circuit representation of modular LFSR is shown in Figure 6. The difference in modular LFSR compared to standard LFSR is due to the positions of the XOR gates in the feedback function; in modular LFSR XOR gates are in between adjacent flip-flops. Modular LFSR works faster than standard LFSR, because it has at most one XOR gate between adjacent flip-flops, while there can be several levels of XOR gates in the feedback of standard LFSR. .

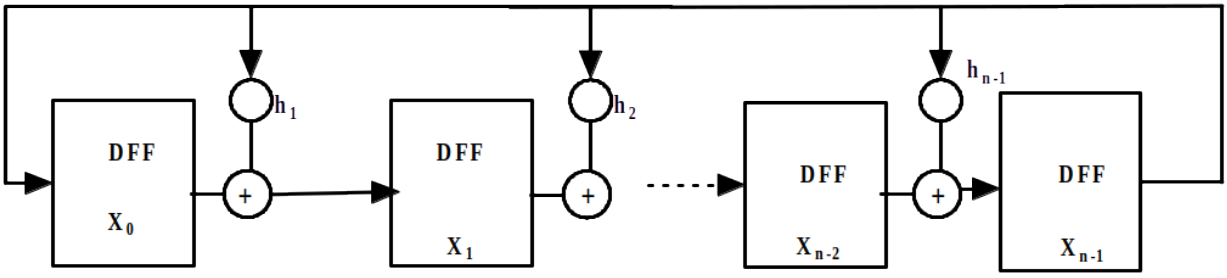

Figure 5. Modular LFSR

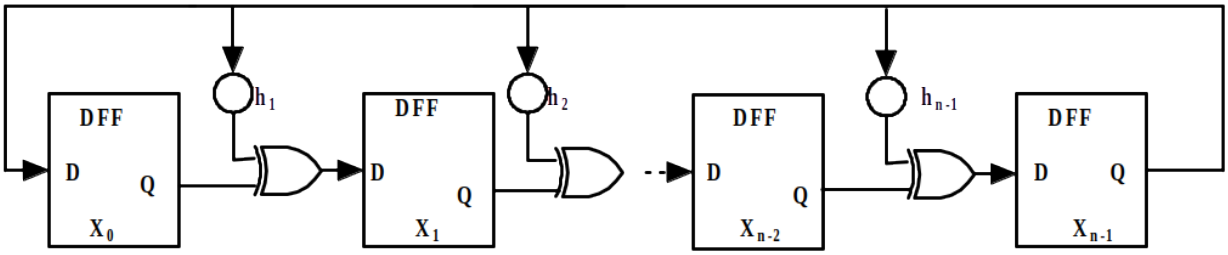

Figure 6. Circuit representation of modular LFSR

In modular LSFR the output of any flip-flop may or may not participate in the XOR function; if output of any flip-flop X_i say, provides input to the XOR gate which feeds the input of flip-flop X_{i+1} then corresponding tap point h_i (Figure 5) is 1. In the circuit representation (Figure 6) of h_i=1, then there is an XOR gate from output of flip-flop X_i to input of flip-flop X_{i+1}; else output of flip-flop X_i is directly fed to input of flip-flop X_{i+1}.

Similar to standard LFSR, a properly-designed modular LFSR can generate a near-exhaustive set of patters, as it can cycle through distinct 2^n - 1 states (except 0s is all flip-flops).

The following matrix system of equations describes the modular LFSR.

$$\begin{bmatrix} X_0(t+1) \\ X_1(t+1) \\ X_2(t+1) \\ \ldots\ldots\ldots\ldots \\ X_{n-2}(t+1) \\ X_{n-1}(t+1) \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0\ldots\ldots 0 & 1 \\ 1 & 0 & 0\ldots\ldots 0 & h_1 \\ 0 & 1 & 0\ldots\ldots 0 & h_2 \\ \multicolumn{4}{c}{\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots} \\ 0 & 0 & 0\ldots\ldots 0 & h_{n-2} \\ 0 & 0 & 0\ldots\ldots 1 & h_{n-1} \end{bmatrix} \begin{bmatrix} X_0(t) \\ X_1(t) \\ X_2(t) \\ \ldots\ldots\ldots \\ X_{n-2}(t) \\ X_{n-1}(t) \end{bmatrix}$$

This LFSR given the matrix can be written as $X(t+1) = T_S X(t)$. In this case $X_0(t+1) = X_{n-1}(t)$, which implies that X_{n-1} directly feedbacks X_0. $X_1(t+1) = X_0(t) + h_1 X_{n-1}(t)$, which implies that depending on h_1=0 (or 1), input to X_1 is X_0 (or X_0 XORed with output of X_{n-1}). Similar logic holds for inputs to all flip-flops from X_1 to X_{n-1}.

This LFSR can also be described by the characteristic polynomial:

$$f(x) = 1 + h_1 x + h_2 x^2 + \ldots h_{n-2} x^{n-2} + h_{n-1} x^{n-1} + x^n$$

Now, we give an example of a modular LFSR and illustrate the pattern generated by the register. Figure 7 shows an example of a standard LFSR.

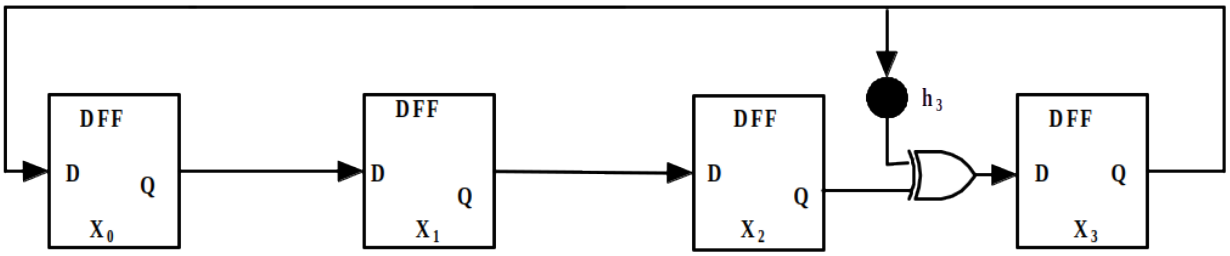

Figure 7. Example of a modular LFSR

The matrix of the LFSR is as follows

$$\begin{bmatrix} X_0(t+1) \\ X_1(t+1) \\ X_2(t+1) \\ X_3(t+1) \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} X_0(t) \\ X_1(t) \\ X_2(t) \\ X_3(t) \end{bmatrix}$$

The characteristic polynomial of the LFSR is $f(x) = 1 + x^3 + x^4$.

If the initial values of the flip-flops are $X_0 = 1, X_1 = 0, X_2 = 0, X_3 = 0$ then the sequence of patters is as follows:

$$\begin{bmatrix} X_0 \\ X_1 \\ X_2 \\ X_3 \end{bmatrix} \begin{matrix} \\ \\ :: \\ \\ \end{matrix} \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

So the LFSR generates 15 patterns (excluding all 0s) after which a pattern is repeated. It may be noted that this LFSR generates all patters (except all 0s) which is generated by a 4 bit counter.

Now, the question arises, whether any LFSR would generate all $2^n - 1$ patters? The answer is no. Only for a few characteristic polynomials the LFSR is maximal length; such polynomials are called primitive polynomials (List of such polynomials can be found in Bardell et al. [2]).

4. Hardware response compactor

As discussed in Section 2 of this module, expected output (i.e., golden response) of the CUT cannot be sorted explicitly in a memory and compared with response obtained from the CUT. In other words, in BIST, it is necessary to compress the large number of CUT responses to a manageable size that can be stored in a memory and compared. In response compaction, sometimes it may happen that the compacted response of the CUT under normal and failure conditions are same. This is called aliasing during compaction.

In this section we will discuss some simple techniques to compress CUT responses namely (i) number of 1s in the output and (ii) transition count at the output. For other complex techniques like LFSR based compaction, multiple input signature register based compaction, built-in logic observer based compaction etc. the reader is referred to [3,4, 5].

4.1 Number of 1s compaction

Number of 1s compaction, is a very simple technique where we count the number of ones in the output responses from the CUT. Figure 8 shows a simple example of such a compaction. In Figure 8 (a), the CUT under normal condition is shown where the inputs are given by the LFSR of Figure 4. It may be noted that 7 patterns (non-similar) were generated by the LFSR, which when given as input to the CUT generates output as 0001000, making number of 1s as 1. Figure 8 (b) shows the same circuit with s-a-1 fault. When the same inputs are given to the CUT the output is 0001100, making number of 1s as 2. So fault can be detected by the compaction as there is difference in number of 1s at the output of the CUT for the given input patterns. In other words, for the input patterns (from the LFSR), “number of 1s” based compaction is not aliasing. It may be noted that corresponding to the input patters, value of 1 is stored (as golden signature) in the memory which is compared with compacted response of the CUT.

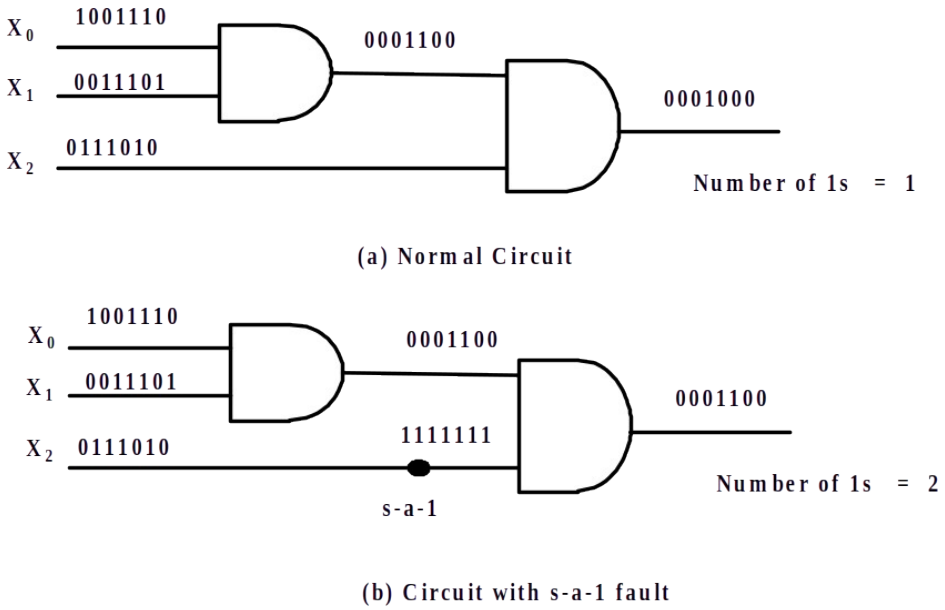

Figure 8. Example of response compaction “number of 1s” : non-aliasing

Figure 9 shows another simple example of number of 1s compaction. In Figure 9 (a), the CUT under normal condition is shown where the input in given by the LFSR of Figure 4. The CUT generates output as 0000110, making number of 1s as 2. Figure 8 (b) shows the same circuit with s-a-1 fault. When the same inputs are given to the CUT the output is 1000100, making number of 1s as 2. So fault cannot be detected by the compaction; the number of 1s at the output of the CUT for the given input patterns is same under normal and s-a-1 conditions. In other words, for the input patterns (from the LFSR), “number of 1s” based compaction is aliasing.

In the next subsection we will study these two circuits using transition count based compaction technique.

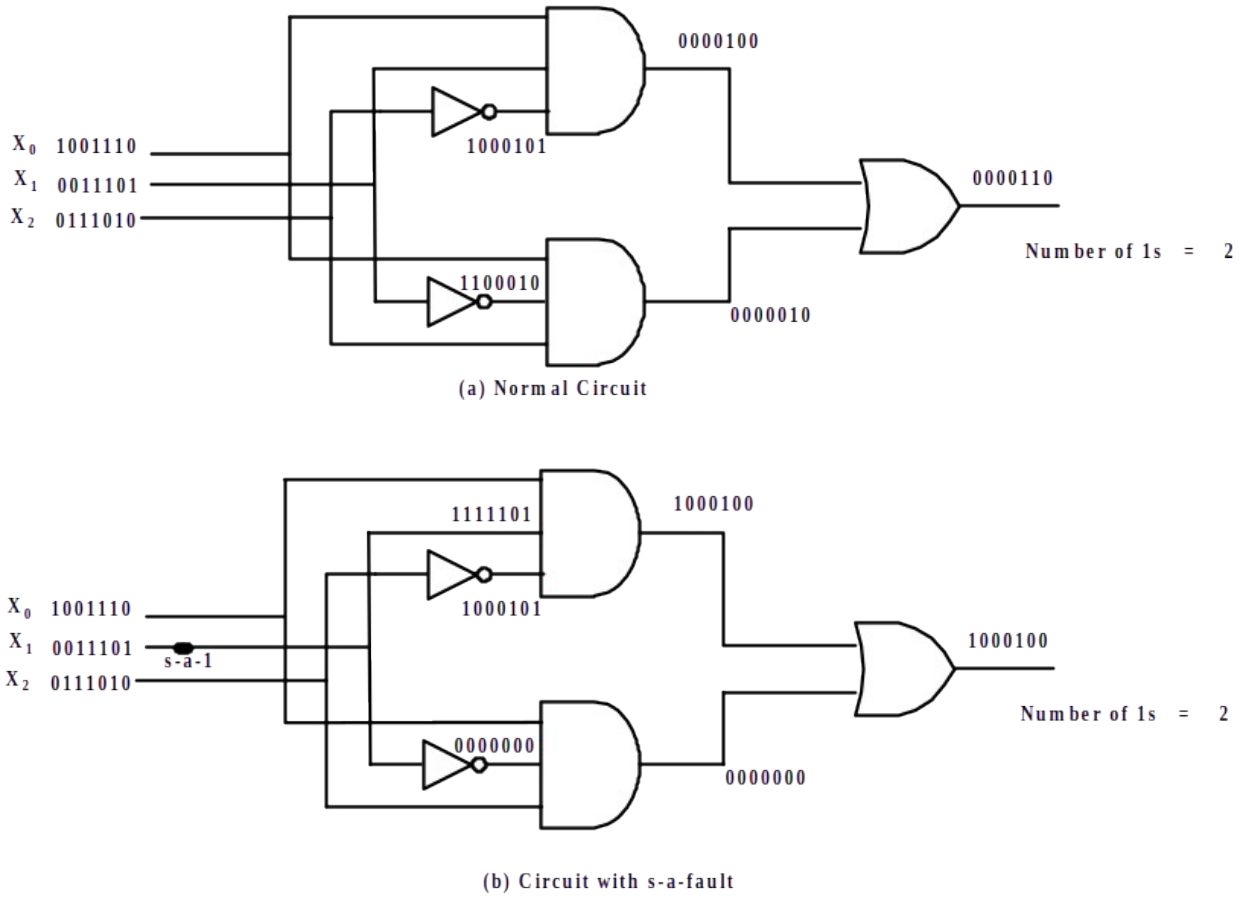

Figure 9. Example of response compaction “number of 1s” -aliasing

4.2 Transition count response compaction

In this method of response compaction the number of transitions from 0 to 1 and 1 to 0 at outputs of the CUT are counted. Figure 10 shows the same circuit of Figure 8, when compaction technique is “transition count”. In Figure 10 (a), the CUT under normal condition is shown where the inputs are given by the LFSR of Figure 4. The CUT generates output as 0001000, making transition count as 2; in the output sequence there is a transition from 0 to 1 and then from 1 to 0. Figure 8 (b) shows the same circuit with s-a-1 fault. When the same inputs are given to the CUT the output is 0001100, making transition count as 2. So fault cannot be detected by the compaction.

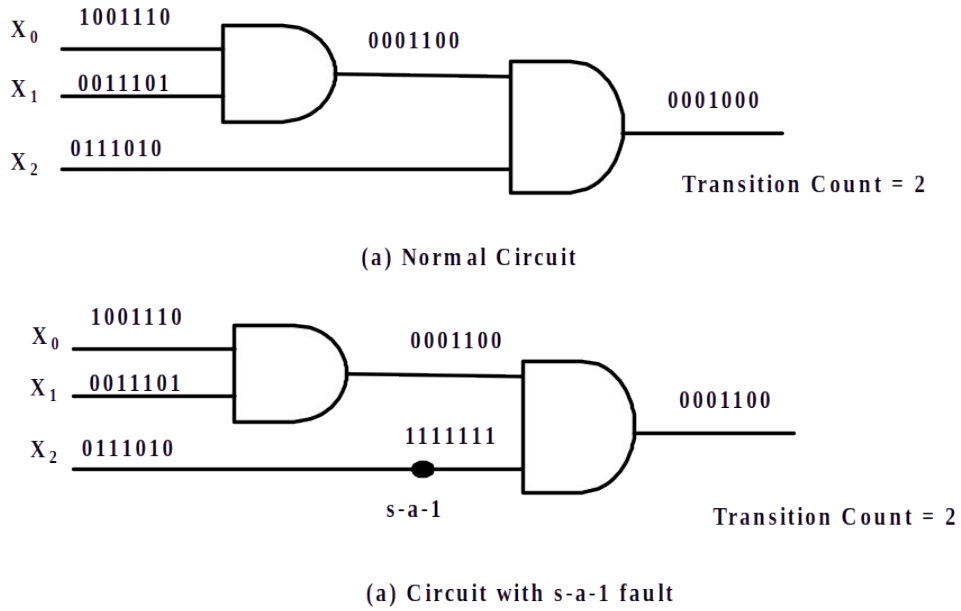

Figure 10. Example of response compaction “transition count” -aliasing

Figure 11 shows the same circuit of Figure 9 when the “transition cout” based compaction is used. In Figure 11 (a), the CUT under normal condition is shown where the input is given by the LFSR of Figure 4. The CUT generates output as 0000110, making transition count as 2. Figure 11 (b) shows the same circuit with

s-a-1 fault. When the same inputs are given to the CUT the output is 1000100, making transition count as 3. So fault is detected by the compaction.

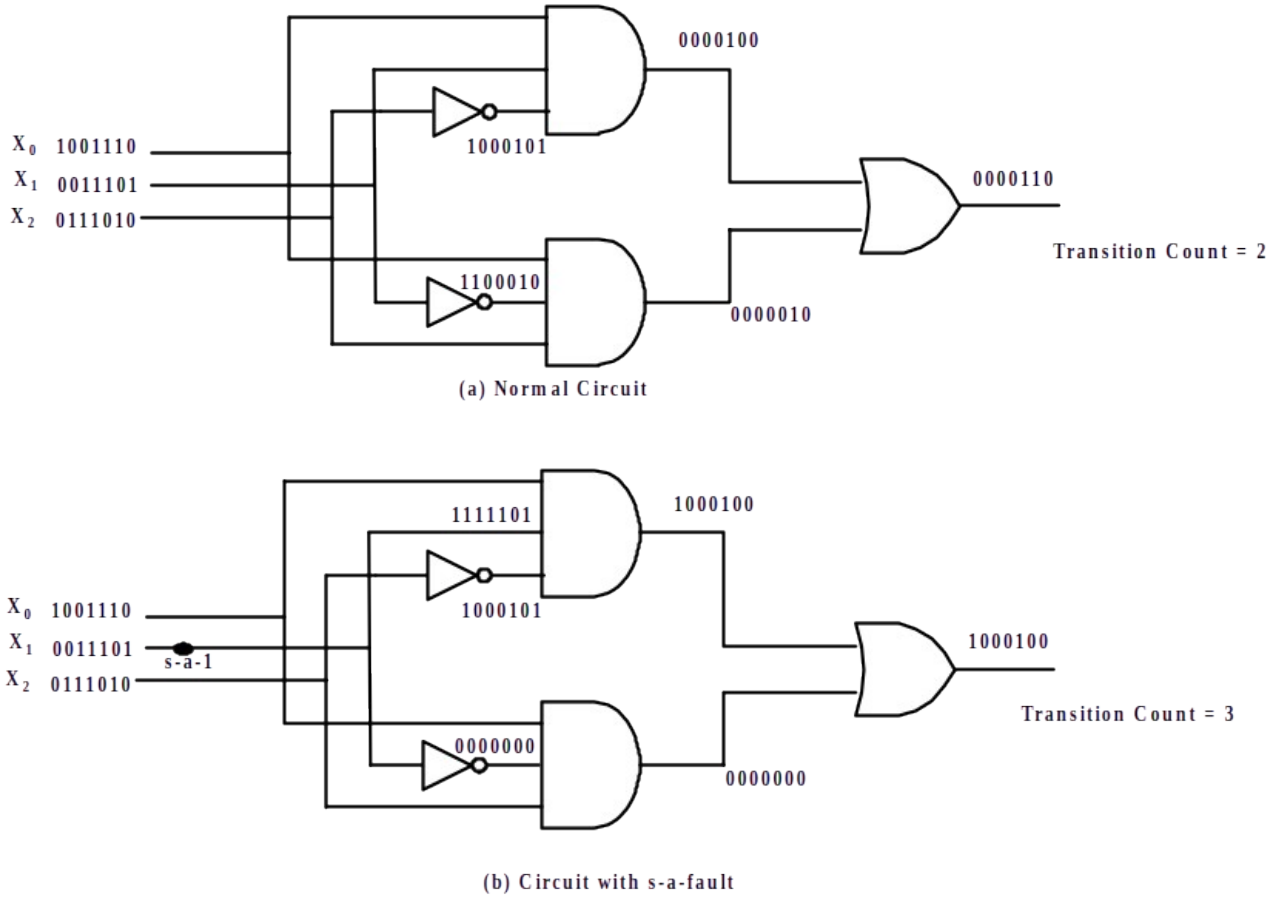

Figure 11. Example of response compaction “transition count—non-aliasing

5. Conclusions

In this lecture we have seen that modern day ICs may develop faults even after manufacturing test. So additional testing is required for assuring quality of service. BIST is such a test procedure which facilitates testing of circuits before every time they start their operations. In this module we have discussed most of the important components of BIST.

Till now, in this course we were discussing about testing of digital circuits comprising Boolean gates and flip-flops. However, memory is also a very vital element for digital circuits. The structure of memory blocks are basically different compared to logic blocks. In the next lecture we will deal with testing of memory blocks.

References

1. Mano, M. Morris,. Digital Design, 2/e. Prentice-Hall of India. 1995.
2. P. H. Bardell, W. H. McAnney, and J. Savir, Built-In Test for VLSI: Pseudorandom Techniques. New York: John Wiley and Sons, Inc., 1987.
3. R. A. Frohwerk, “Signature Analysis: A New Digital Field Service Method,” Hewlett-Packard Journal, vol. 28, no. 9, pp. 2–8, May 1977.
4. S. Z. Hassen and E. J. McCluskey, “Increased Fault Coverage Through Multiple Signatures,” in Proc. of the International Fault-Tolerant Computing Symp., June 1984,pp. 354–359.
5. M. Bushnell and V.D. Agrawal, “Essentials of Electronic Testing for Digital, Memory & Mixed-Signal Circuits”, Kluwer Academic Publishers, 2000

Questions and Answers

1. In BIST how do we know which LFSR is to be used pattern generation?

Answer:

Firstly, (obviously) the width of the LFSR is same as that of the number of inputs of the circuit.

Secondly, the characteristic polynomial of the LFSR should be primitive polynomial. This ensures that the LFSR generates all possible patterns (except all 0s).

Thirdly, the seed will decide the sequence of patterns. By fault simulation the test patterns can be decided. Following that a proper compaction technique with minimal aliasing property for the circuit needs to be chosen. Then the seed is to be chosen in a manner so that the required test patterns among all the patterns generated by the LFSR should be generated initially. In other words, the seed should be chosen such that patterns of LFSR which do not test any faults (or any new faults compared to previous patterns) should be generated at latter phases. When all the required patterns have been generated and all faults have been tested the LFSR can be stopped (without requirement to generate the full cycle of patterns).

2. Why LFSR cannot have all 0 state?

Answer:

If the seed is all 0 state, then the LFSR will be stuck at all 0 state as the feedback logic is XOR gates.